

Análise de Dados com Python

Turmas 03 e 04 - Situação de Aprendizagem (Projeto Avaliativo) -
Módulo 1 - Semana 13 e 14

SUMÁRIO

1 CONTEXTUALIZAÇÃO	1
2 DESAFIO	1
3 RESULTADOS ESPERADOS (ENTREGA)	2
4 REQUISITOS DA APLICAÇÃO	2
5 ROTEIRO DA APLICAÇÃO	3
5.1 FORMATO DO PIPELINE	3
5.2 ETAPAS DO PIPELINE	3
5.3 ARQUITETURA PROPOSTA	4
5.4 DICIONÁRIO DE DADOS DA CAMADA SILVER	4
5.5 CONSTRAINTS	6
5.6 CONVERSÕES DE TRATAMENTO DE DADOS	7
5.7 DOCUMENTAÇÃO NO README.MD	7
5.8 USO DO GITHUB	7
6 CRITÉRIOS DE AVALIAÇÃO	7

1 CONTEXTUALIZAÇÃO

As empresas de comércio geram dados diariamente por meio de suas vendas. Cada venda pode conter informações sobre a filial, a cidade, a categoria do produto, o preço, a quantidade comprada, a forma de pagamento, a data e o valor total da operação.

Esses dados ajudam uma empresa a compreender seu desempenho e apoiar decisões importantes, como:

- Identificar a filial com maior faturamento;
- Verificar quais categorias de produtos vendem mais;
- Conhecer a forma de pagamento mais utilizada;
- Calcular o valor médio das vendas;
- Comparar o desempenho das filiais;
- Identificar os períodos com maior quantidade de vendas;
- Avaliar a satisfação dos clientes.

Entretanto, os dados nem sempre estão prontos para análise. Eles podem estar armazenados em arquivos CSV, apresentar nomes de colunas pouco padronizados, conter tipos de dados incorretos ou exigir conversões de datas e valores numéricos.

Neste projeto, será desenvolvido um fluxo introdutório de análise de dados utilizando PostgreSQL, SQL, Python, Pandas, estatística descritiva e controle de versão com Git e GitHub, seguindo uma estrutura inspirada na Arquitetura Medallion (Raw e Tratada).

2 DESAFIO

Uma rede de supermercados deseja analisar os registros de vendas realizados em suas filiais durante determinado período. A empresa possui uma base de dados pública com informações de vendas, mas precisa organizar os dados para responder perguntas simples de negócio.

Sua missão é construir, do zero, um fluxo estruturado que armazene os dados brutos no PostgreSQL, realize consultas e exportações, e utilize o Python para tratar, analisar e responder perguntas fundamentais de negócio de forma clara e objetiva.

3 RESULTADOS ESPERADOS (ENTREGA)

Ao final do projeto, você terá construído um fluxo completo que:

- Baixa e armazena os dados brutos fielmente no PostgreSQL (Camada Raw), garantindo rastreabilidade;
(<https://www.kaggle.com/datasets/faresashraf1001/supermarket-sales>)
- Limpa e organiza os dados no Python/Pandas (Camada Tratada), com tipagem correta e integridade estrutural;
- Responde a perguntas de negócio reais apoiadas por métricas e visualizações que tornam a informação acessível para a tomada de decisão.

Perguntas de negócio a serem respondidas:

- Qual filial apresentou o maior faturamento?
- Qual filial realizou a maior quantidade de vendas?
- Qual linha de produto apresentou o maior faturamento?
- Qual linha de produto recebeu a melhor avaliação média?
- Qual foi a forma de pagamento mais utilizada?
- Qual foi o valor médio das vendas?
- Qual foi a maior venda registrada?
- Em qual dia da semana ocorreu a maior quantidade de vendas?

Não serão aceitos projetos submetidos após a data limite da atividade, e, ou alterados depois de entregues.

Importante:

1. Será considerado como data final de entrega a última atualização no repositório do projeto no GitHub. Lembre-se de não modificar o código até receber sua nota.
2. Não esqueça de submeter o link no AVA. O link não submetido terá penalidade na nota.
3. Deverá ser disponibilizado o[s] link[s] na tarefa **Módulo 1 - Projeto Avaliativo**, presente na semana 13 do AVA até o dia **05/10/2026 às 22h.**

4 REQUISITOS DA APLICAÇÃO

Abaixo seguem os requisitos a serem desenvolvidos, organizados pelas fases do projeto:

- Fase 0 - Banco e tabelas (0_criar_banco.sql e 02_criar_tabelas.sql): criar o banco de dados e as tabelas necessárias (Raw e Tratada), aplicando chaves primárias e restrições básicas (NOT NULL, CHECK).
- Fase 1 - Extração e Carga Raw: carregar os dados brutos no PostgreSQL sem alterar o conteúdo original.
- Fase 2 - Consultas e Exportação: realizar consultas SQL utilizando comandos fundamentais (SELECT, WHERE, GROUP BY, SUM, AVG, etc.) e exportar os dados para CSV.
- Fase 3 - Transformação com Pandas: limpar, tipar, verificar valores ausentes, remover duplicidades e criar colunas derivadas no Python.
- Fase 4 - Análise e Gold: responder às perguntas de negócio com estatística descritiva e gráficos claros.
- Boas práticas: credenciais protegidas, .gitignore adequado e código organizado (PEP-8 e modularização).

5 ROTEIRO DA APLICAÇÃO

5.1 FORMATO DO PIPELINE

O projeto é um fluxo de dados em Python + PostgreSQL, estruturado em pastas padronizadas:

- sql/ (Scripts de banco e consultas)
- src/ (Scripts Python de leitura, ETL e estatística)
- data/raw/ (Dados originais e exportados)
- data/processed/ (Dados tratados)
- resultados/ (Estatísticas e gráficos)

5.2 ETAPAS DO PIPELINE

ARQUIVO	ESTADO	DESCRIÇÃO
sql/01_criar_banco.sql	Entregar	Criação do banco de dados no PostgreSQL
sql/02_criar_tabelas.sql	Entregar	Criação das tabelas Raw e Tratada com restrições

sql/03_consultas.sql	Entregar	Consultas SQL fundamentais e exportação para CSV
src/01_leitura_dados.py	Entregar	Leitura e inspeção inicial do CSV no Pandas
src/02_etl_vendas.py	Entregar	Limpeza, tipagem, tratamento de nulos e colunas derivadas
src/03_estatistica.py	Entregar	Estatística descritiva, respostas de negócio e gráficos
README.md	Entregar	Documentação completa da solução no GitHub
requirements.txt	Entregar	Dependências e bibliotecas do projeto
.gitignore	Entregar	Arquivos e pastas ignorados pelo Git

5.3 ARQUITETURA PROPOSTA

CAMADA	O QUE É	ONDE VAI FICAR
Raw (Bruta)	Cópia fiel dos dados originais do CSV importados para o banco.	Tabela raw_vendas / data/raw/
Tratada	Dados limpos, tipados (NUMERIC, DATE) e com colunas derivadas no Pandas.	data/processed/vendas_tratadas.csv
Resultados	Métricas de negócio, estatísticas descritivas e visualizações gráficas.	resultados/

Fonte dos dados: [Dataset público Supermarket Sales \(Kaggle\)](#), delimitado por vírgula, com valores numéricos e datas que exigem tratamento adequado.

6 DICIONÁRIO DE DADOS DA CAMADA TRATADA

COLUNA	TIPO ESPERADO	RESTRIÇÕES / DESCRIÇÃO
id_venda	VARCHAR(50)	PRIMARY KEY, NOT NULL
Filial	VARCHAR(10)	NOT NULL
Cidade	VARCHAR(100)	NOT NULL
tipo_cliente	VARCHAR(50)	—
Gênero	VARCHAR(20)	—
linha_produto	VARCHAR(150)	NOT NULL

preco_unitario	NUMERIC(10,2)	CHECK >= 0
Quantidade	INTEGER	CHECK > 0
Imposto	NUMERIC(10,2)	CHECK >= 0
valor_total	NUMERIC(12,2)	CHECK >= 0 (Calculado / Conferido)
data_venda	DATE	Convertido de texto para data
hora_venda	TIME	Convertido para horário
forma_pagamento	VARCHAR(50)	NOT NULL
custo_mercadoria	NUMERIC(12,2)	CHECK >= 0
margem_percentual	NUMERIC(10,2)	—
receita_bruta	NUMERIC(12,2)	CHECK >= 0
Avaliação	NUMERIC(4,2)	CHECK entre 0 e 10

7 CRITÉRIOS DE AVALIAÇÃO

A tabela abaixo apresenta os critérios que serão avaliados durante a correção do projeto.

Ele possui variação de nota de 0 (zero) a 10 (dez) como nota mínima e máxima.

Serão desconsiderados e atribuída a nota 0 (zero) os projetos que apresentarem plágio de soluções encontradas na internet ou de outros colegas.

Lembre-se: Você está livre para utilizar outras soluções como base, mas não é permitida a cópia.

Uso do GitHub e Readme.md				
Nº	Critério de Avaliação	0	0,5	0,75
1	Versionamento com branches e commits	Não apresentou branches ou commits organizados.	Utilizou os prefixos do <i>Conventional Commits</i> apenas em parte dos commits, deixando outros commits sem padronização ou com nomes genéricos.	O repositório do projeto apresentou um histórico de commits estruturados, com mensagens em inglês/português imperativo, prefixos padronizados (Conventional Commits) e escopo bem definido para cada alteração realizada.
2	Organização dos arquivos no repositório	Não apresentou estrutura de pastas.	Organizou parcialmente a estrutura de diretórios do projeto, criando a maioria das pastas requeridas (sql/, src/, data/ ou resultados/), porém apresenta inconsistências na padronização, no versionamento (ex.: commits pouco claros ou arquivos desnecessários	Organizou a estrutura de diretórios do projeto (sql/, src/, data/ e resultados/), padronizando o versionamento e a separação de responsabilidades dos arquivos de acordo com as boas práticas do mercado de software.

			no Git) ou falhas pontuais na separação de responsabilidades dos arquivos.	
3	README.md e documentação	Sem documentação.	A documentação do projeto foi criada via README.md, porém está incompleta ou omite elementos essenciais, apresentando apenas parte das informações exigidas — como descrição superficial das tecnologias, instruções insuficientes para a execução da aplicação ou ausência da demonstração dos resultados obtidos.	A documentação do projeto foi criada via README.md, contendo a descrição das tecnologias utilizadas, as instruções detalhadas para execução da aplicação e a demonstração dos resultados obtidos, tecnologias e resultados.
Nº	Critério de Avaliação	0	0,25	0,5
4	Consultas SQL e exportação para CSV	Não realizou consultas ou exportação.	Consultas parciais ou sem exportar CSV.	Construiu e executou consultas SQL para extração e exportação de dados, realizando a ingestão no ambiente Python para leitura e análise exploratória de dados (EDA) com a biblioteca Pandas.
5	Leitura do CSV e DataFrame	Não realizou a leitura no Python.	Leitura simples sem inspeção de nulos.	Utilizou a biblioteca Pandas para importação de dados e realizou a verificação estrutural completa, identificando tipos de variáveis, inconsistências e estatísticas descritivas básicas.

Nº	Critério de Avaliação	0	0,75
6	Banco de dados e importação Raw	Não criou o banco ou a tabela Raw.	Configurou o banco de dados PostgreSQL e realizou a importação do conjunto de dados fornecido, utilizando scripts SQL/ferramentas de SGBD conforme as especificações técnicas da atividade.

Nº	Critério de Avaliação	0	0,5	1,0
7	ETL, limpeza e transformação	Não realizou o tratamento dos dados.	Realizou a conversão dos tipos de dados ou o tratamento de nulos, mas com inconsistências nas colunas derivadas ou sem tratar casos limítrofes.	Realizou a conversão e adequação dos tipos de dados (<i>casting</i>), aplicou técnicas para identificação e tratamento de valores ausentes/nulos e construiu novas colunas derivadas (<i>feature engineering</i>) utilizando a biblioteca

				Pandas em ambiente Python.
8	Estatística e perguntas de negócio	Não respondeu às perguntas de negócio propostas	Formulou hipóteses e respondeu às perguntas de negócio propostas, mas limitou a validação a respostas descritivas sem fundamentá-las adequadamente em gráficos, métricas ou estatísticas descritivas extraídas via Pandas.	Formulou hipóteses e respondeu às perguntas de negócio propostas, validando os resultados com gráficos, estatísticas descritivas e métricas extraídas via bibliotecas do Python (como Pandas).
Nº	Critério de Avaliação	0	1,5	2,0
9	Modelagem e constraints	Sem modelagem ou restrições.	Construiu o script SQL do banco de dados definindo a Chave Primária (PRIMARY KEY), mas aplicou de forma parcial ou inconsistente as restrições de obrigatoriedade (NOT NULL) e validação (CHECK), deixando de contemplar regras de negócio do domínio.	O script SQL do banco de dados foi construído implementando considerando as restrições de integridade, identificando a Chave Primária (PRIMARY KEY), garantindo a obrigatoriedade dos campos críticos (NOT NULL) e aplicando regras de validação de dados (CHECK) de acordo com os requisitos do domínio.
10	Salvamento da base tratada	A base tratada não foi salva.	A base de dados tratada foi salva no diretório data/processed/, porém o arquivo não segue rigorosamente o padrão de nomenclatura estabelecido na documentação ou o arquivo de dicionário de dados correspondente foi omitido, está incompleto ou desatualizado.	A base de dados tratada foi salva no diretório data/processed/, seguindo o padrão de nomenclatura definido na documentação e acompanhada pelo arquivo de dicionário de dados correspondente.